AMAZON S3: CREACIÓN DE UN SITIO WEB ESTÁTICO ALOJADO EN AMAZON S3 UTILIZANDO VISUAL STUDIO CODE

Objetivo:

En la era digital actual, la creación de sitios web estáticos es una tarea fundamental en el desarrollo web. Los sitios web estáticos son eficientes, seguros y adecuados para proyectos que no requieren interacción en tiempo real con bases de datos o contenido dinámico. Una forma popular de alojar un sitio web estático es utilizando Amazon S3 (Simple Storage Service), un servicio de almacenamiento en la nube altamente escalable y confiable ofrecido por Amazon Web Services (AWS).

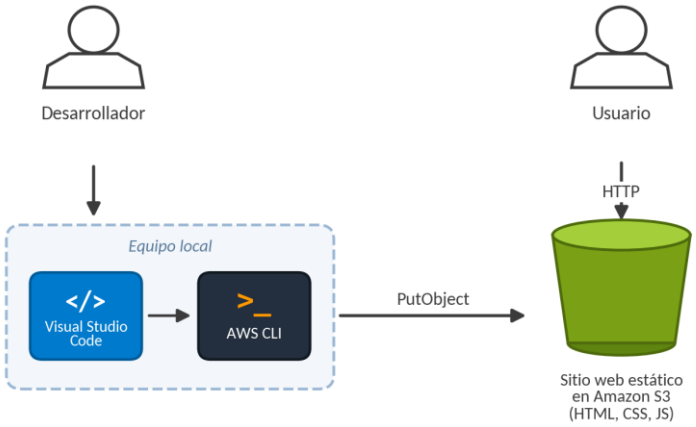
A lo largo de esta práctica, aprenderemos a configurar un proyecto web estático, utilizar **Visual Studio Code** en nuestro propio equipo junto con la **AWS CLI** para desarrollarlo y, finalmente, implementar nuestro sitio en un bucket de S3, aprovechando así la potencia de la nube para crear y alojar aplicaciones web de manera eficiente y escalable.

Requerimientos:

- Disponer de acceso a los recursos de AWS a través de un *sandbox* de AWS Academy (Learner Lab).
- Disponer de un equipo (Windows, macOS o Linux) con conexión a Internet y tener instaladas las siguientes herramientas:

Herramienta	Comprobar instalación (en un terminal)	Descarga
Visual Studio Code	`code --version`	https://code.visualstudio.com
Git	`git --version`	https://git-scm.com/downloads
AWS CLI v2	`aws --version`	https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html

Arquitectura propuesta:



Adaptado de J. E. Vera · CC BY-NC

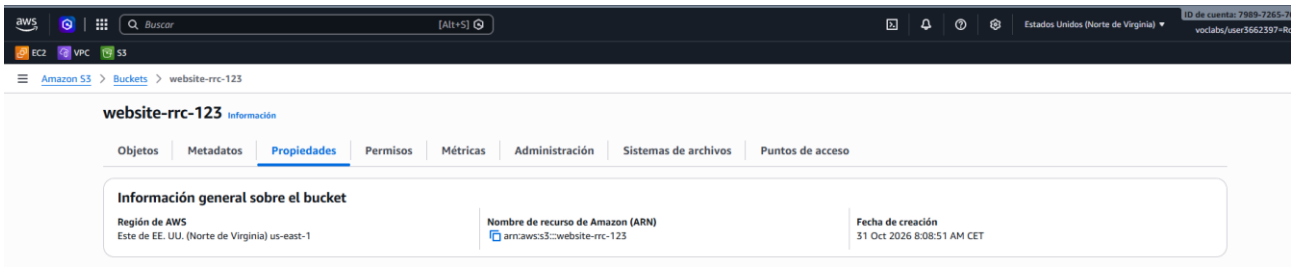
Realización:

1) El primer paso por realizar es crear el bucket de Amazon S3 que alojará el contenido web estático. Para ello, se accede al servicio de Amazon S3 desde la consola de administración de AWS (botón verde de AWS en el Learner Lab), se selecciona en el menú lateral la opción **Buckets** y se presiona el botón **Create bucket**.

2) En el siguiente formulario se completa introduciendo:

- **Bucket name**: *website-<iniciales-nombre-apellidos>-1234*. Es fundamental que el nombre del bucket sea único a nivel global
- **AWS Region**: Seleccionar la opción *us-east-1 (N. Virginia)* o *us-west-2 (Oregon)*, ya que son las únicas permitidas en AWS Academy Learner Labs.
- **Block Public Access settings for this bucket**. Desmarcar la opción *Block all public access*, ya que nuestro bucket deberá ser accesible públicamente al tratarse de un sitio web. Además, marcar la casilla de aceptación del reconocimiento que supone hacer público un bucket de S3.

Por último, presionar el botón **Create bucket**.



3) A continuación, preparamos el entorno de trabajo en nuestro equipo. Abrimos **Visual Studio Code** y, desde el menú **Terminal › New Terminal**, abrimos un terminal integrado. Comprobamos que las tres herramientas están instaladas ejecutando los comandos de la tabla de requerimientos (`code --version`, `git --version` y `aws --version`).

4) Configuramos la **AWS CLI** para que utilice las credenciales de nuestro Learner Lab. En AWS Academy, con el laboratorio iniciado (botón **Start Lab** y círculo verde junto a la palabra *AWS*), se presiona el botón **AWS Details** y, junto a **AWS CLI**, el botón **Show**. Aparecerá un bloque de texto como el siguiente:

```
[default]
aws_access_key_id=ASIA................
aws_secret_access_key=...................
aws_session_token=............................
```

Ese bloque debe copiarse en el fichero `credentials` de la carpeta `.aws` de nuestro directorio personal. Desde el terminal de VS Code, se crea la carpeta (si no existe) y se abre el fichero en el editor:

Sistema	Comandos
Windows (PowerShell)	`New-Item -ItemType Directory -Force $HOME\.aws` `code $HOME\.aws\credentials`
Linux / macOS	`mkdir -p ~/.aws` `code ~/.aws/credentials`

Se pega el bloque copiado en el editor y se guarda el fichero (**Ctrl+S**). Después, se indica la región con la que trabajará la CLI:

```
aws configure set region us-east-1
```


Adaptado de J. E. Vera · CC BY-NC

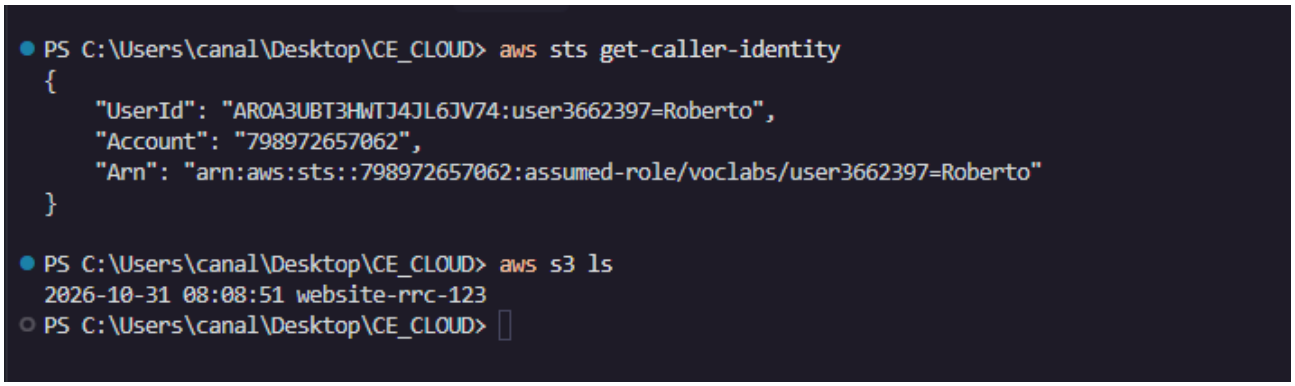
> **Importante:** las credenciales del Learner Lab son **temporales**: caducan al terminar la sesión del laboratorio (aproximadamente 4 horas) y cambian cada vez que se pulsa *Start Lab*. Si en algún momento aparece un error de token caducado, hay que volver a copiar las credenciales y repetir este paso. Nunca deben compartirse ni subirse a un repositorio.

Si el comando `code` no se reconoce en el terminal (suele ocurrir en macOS), se puede abrir el fichero desde **File › Open File…**, o instalar el comando desde la paleta de comandos (**Ctrl+Shift+P** o **Cmd+Shift+P**) con la opción *Shell Command: Install 'code' command in PATH*.

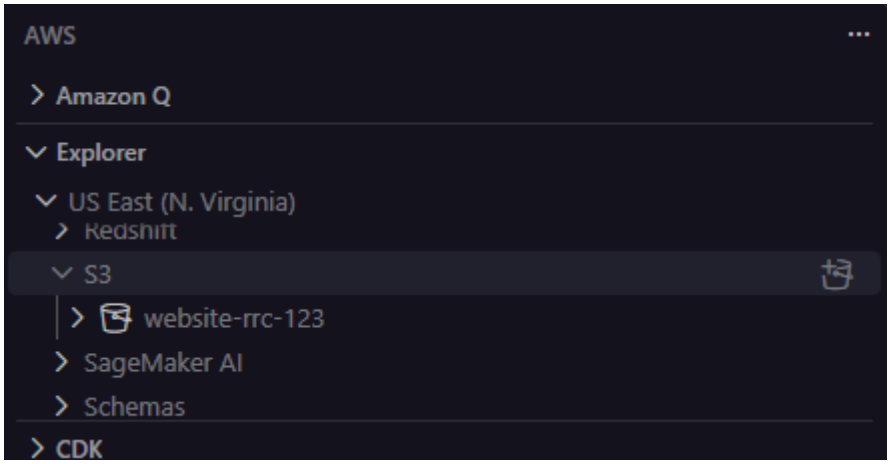
5) Para comprobar que la conexión con AWS funciona, ejecutamos en el terminal:

```
aws sts get-caller-identity
aws s3 ls
```

El primer comando muestra la identidad con la que trabaja la CLI (el campo `Arn` contendrá el rol *voclabs*) y el segundo lista los buckets de la cuenta, entre los que debe aparecer el bucket creado en el apartado 2.



6) **(Opcional)** También se puede comprobar el bucket desde el propio editor mediante la extensión **AWS Toolkit**. Para ello, se abre la vista de extensiones (**Ctrl+Shift+X**), se busca *AWS Toolkit* (publicada por *Amazon Web Services*), se instala y se pulsa el icono de AWS de la barra lateral. Al pedir la conexión, se elige el perfil `default` (el del fichero `credentials`). En el explorador, dentro de la región *us-east-1*, en el apartado **S3** aparecerá el bucket. Si no lo hiciera, hay que refrescar el explorador. La interfaz puede variar según la versión de la extensión.



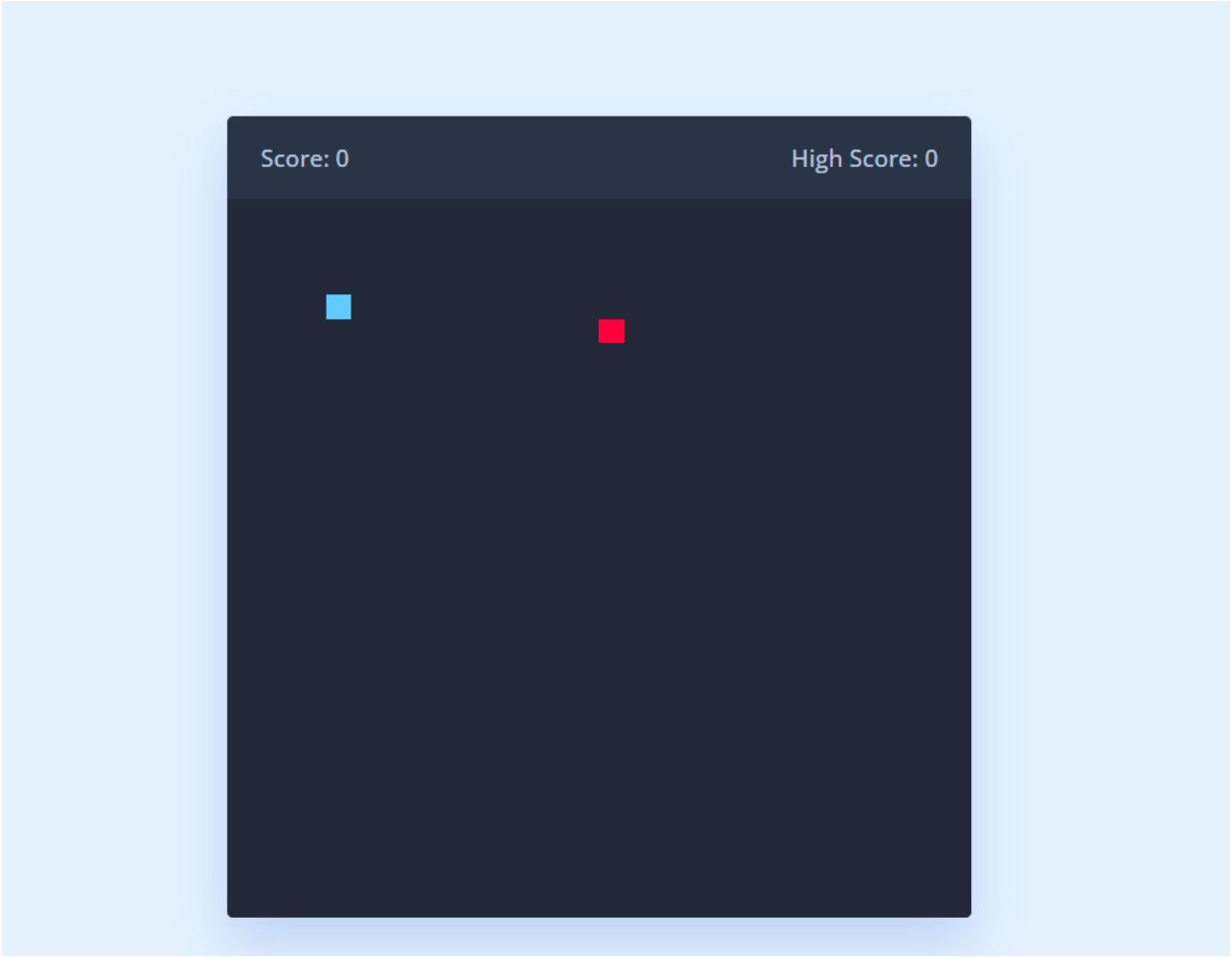
Adaptado de J. E. Vera · CC BY-NC

7) Desde el terminal, clonamos el repositorio donde se encuentra el código del sitio web estático en una carpeta de trabajo (por ejemplo, `Documentos`):

```
git clone https://github.com/jose-emilio/aws-academy-fp-asir.git
```

A continuación, se abre en VS Code la carpeta que contiene el sitio web con **File › Open Folder…** y se selecciona `aws-academy-fp-asir/resources/website-s3-cloud9` (aunque el nombre de la carpeta mencione Cloud9, su contenido es simplemente un sitio web estático). Si VS Code pregunta si se confía en los autores de la carpeta, se acepta. En el explorador de archivos del editor podemos ver los tres archivos del sitio: *index.html*, *script.js* y *style.css*. Se abre un nuevo terminal (**Terminal › New Terminal**) para que se sitúe en esa carpeta.

8) **(Opcional)** Antes de subirlo a la nube, se puede probar el sitio en local abriendo el archivo *index.html* con el navegador (por ejemplo, desde el explorador de archivos del sistema operativo) o con la extensión *Live Server* de VS Code.



9) Copiamos los archivos del sitio web estático en el bucket de S3 creado anteriormente utilizando la AWS CLI, sustituyendo el *placeholder* por el nombre del bucket creado en el apartado 2. El comando se ejecuta desde el terminal situado en la carpeta del sitio web:

```
aws s3 cp . s3://<bucket> --recursive
```

Si el terminal estuviera en la carpeta donde se clonó el repositorio, el comando equivalente sería `aws s3 cp aws-academy-fp-asir/resources/website-s3-cloud9/. s3://<bucket> --recursive`. Cada archivo copiado se muestra en pantalla con una línea `upload: …`. Para comprobar que se han cargado los 3 archivos (index.html, script.js y style.css) se ejecuta:

```
aws s3 ls s3://<bucket>
```


Adaptado de J. E. Vera · CC BY-NC

```
PS C:\Users\canal\Desktop\CE_CLOUD> aws s3 cp '.\Administracion_BBDD_Almacenamiento\T1 - Almacenamiento de Objetos Con Amazon S3\Practica 1.7\aws-academy-fp-asir\resources\website-s3-cloud9\.' s3://website-rrc-123 --recursive
upload: Administracion_BBDD_Almacenamiento\T1 - Almacenamiento de Objetos Con Amazon S3\Practica 1.7\aws-academy-fp-asir\resources\website-s3-cloud9\index.html to s3://website-rrc-123/index.html
upload: Administracion_BBDD_Almacenamiento\T1 - Almacenamiento de Objetos Con Amazon S3\Practica 1.7\aws-academy-fp-asir\resources\website-s3-cloud9\script.js to s3://website-rrc-123/script.js
upload: Administracion_BBDD_Almacenamiento\T1 - Almacenamiento de Objetos Con Amazon S3\Practica 1.7\aws-academy-fp-asir\resources\website-s3-cloud9\style.css to s3://website-rrc-123/style.css
PS C:\Users\canal\Desktop\CE_CLOUD> aws s3 ls s3://website-rrc-123
2026-10-31 08:32:26       1078 index.html
2026-10-31 08:32:26       3550 script.js
2026-10-31 08:32:26       1540 style.css
PS C:\Users\canal\Desktop\CE_CLOUD>
```

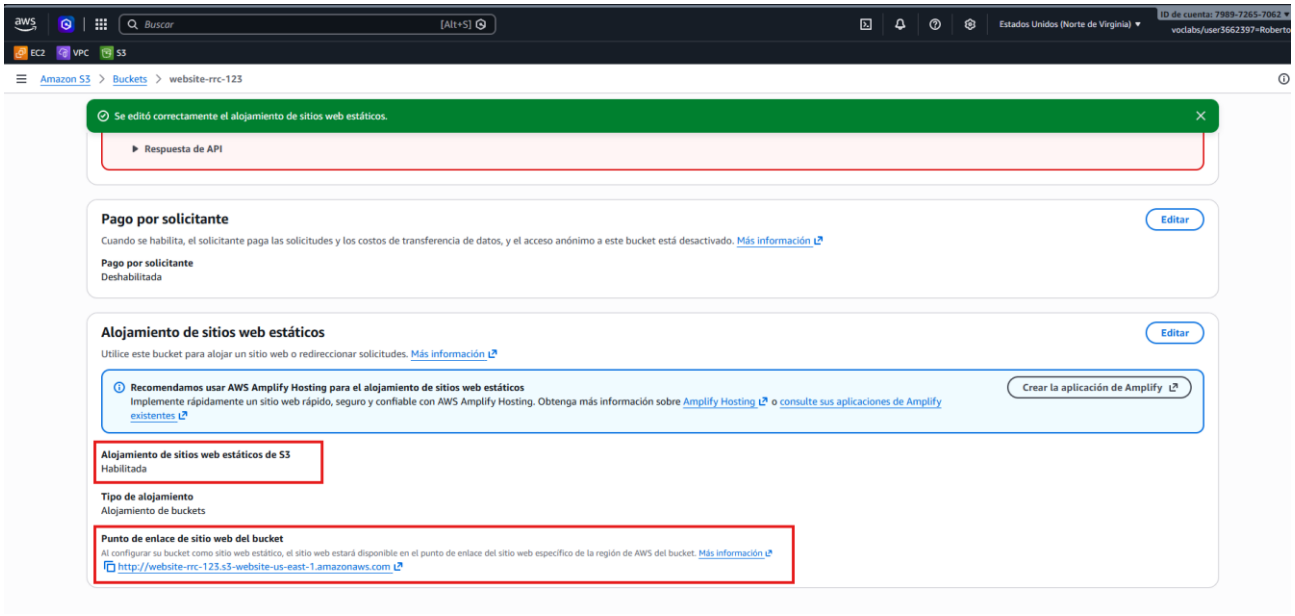
10) A continuación, hay que crear un punto de enlace sobre el bucket de S3 para que actúe como un sitio web estático. Para ello, desde la consola de Amazon S3, en la sección **Buckets**, hacemos clic sobre el nombre del bucket creado y, en la pestaña **Properties**, se hace *scrolling* hasta el final de la página, donde hay un apartado llamado **Static website hosting**. Presionamos la opción **Edit**.

11) En la siguiente ventana seleccionamos la opción **Enable**. A continuación, en la sección **Hosting type** marcamos la opción *Host a static website*, indicando en el apartado **Index document** el nombre de nuestro archivo que se cargará cuando se acceda al sitio web, es decir, *index.html*. Por último, se presiona el botón **Save changes**.

12) Tras ello, volveremos a la pestaña **Properties** del bucket. Si nos desplazamos hacia la sección **Static website hosting** podremos visualizar que, efectivamente, el sitio web del bucket de S3 está habilitado y que además se ha generado un punto de enlace de protocolo HTTP (no HTTPS) de la forma:

http://<nombre-bucket>.s3-website-<region>.amazonaws.com

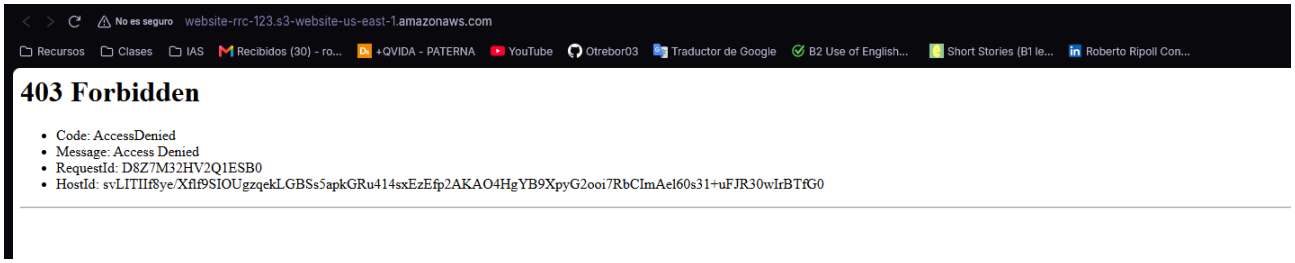
Copiamos la URL anterior en el portapapeles.



13) A continuación, abrimos una ventana del navegador en modo privado/incógnito para comprobar el funcionamiento del sitio web, obteniendo un error **403 Forbidden** (`Code: AccessDenied`).

El error anterior HTTP 403 se debe a que, si bien estamos accediendo al punto de enlace del sitio web del bucket de S3 y el bucket se ha configurado como público, los objetos del bucket no son públicos. Para hacer los objetos del bucket públicos existen varias formas; sin embargo, AWS recomienda que se utilicen las **políticas de buckets** de Amazon S3, ya que permiten un control muy detallado sobre los niveles de permisos necesarios para el acceso a los objetos de los buckets de S3.

Adaptado de J. E. Vera · CC BY-NC



14) Para crear una política de bucket, desde dentro del bucket, presionamos la pestaña **Permissions**. Desde esta pestaña podremos leer en la opción **Permissions overview** que los objetos *pueden* ser públicos (no quiere decir que lo sean, dependerá de la política que creemos). A continuación, avanzaremos hasta la sección **Bucket Policy** y presionamos el botón **Edit**.

15) Las políticas de bucket son documentos escritos en un lenguaje de acceso basado en JSON que permiten especificar el nivel de permisos de acceso a un bucket de S3. Para poder elaborar la política, necesitamos el **ARN** del bucket de S3. El ARN (Amazon Resource Name) de un recurso es un identificador que referencia al recurso globalmente de manera unívoca. Lo podemos encontrar en la parte superior de la ventana de edición, en el campo **Bucket ARN** (tiene la forma `arn:aws:s3:::<bucket>`). Utilizaremos el siguiente documento JSON como política, sustituyendo el *placeholder* por el ARN del bucket:

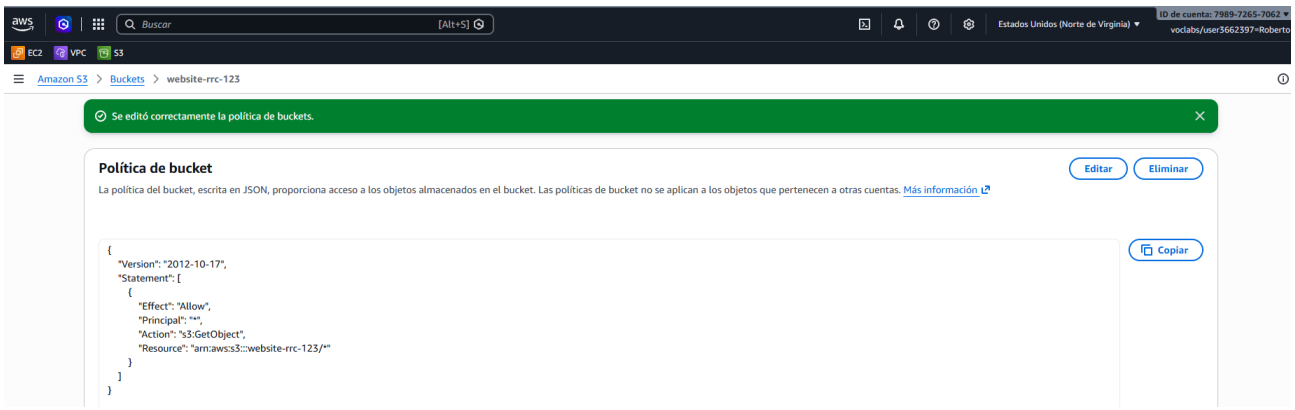
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Principal": "*",
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "<ARN-bucket>/*"
    }
  ]
}
```

Una política de bucket está compuesta por sentencias (*Statements*) donde en cada una de ellas se definen diferentes atributos:

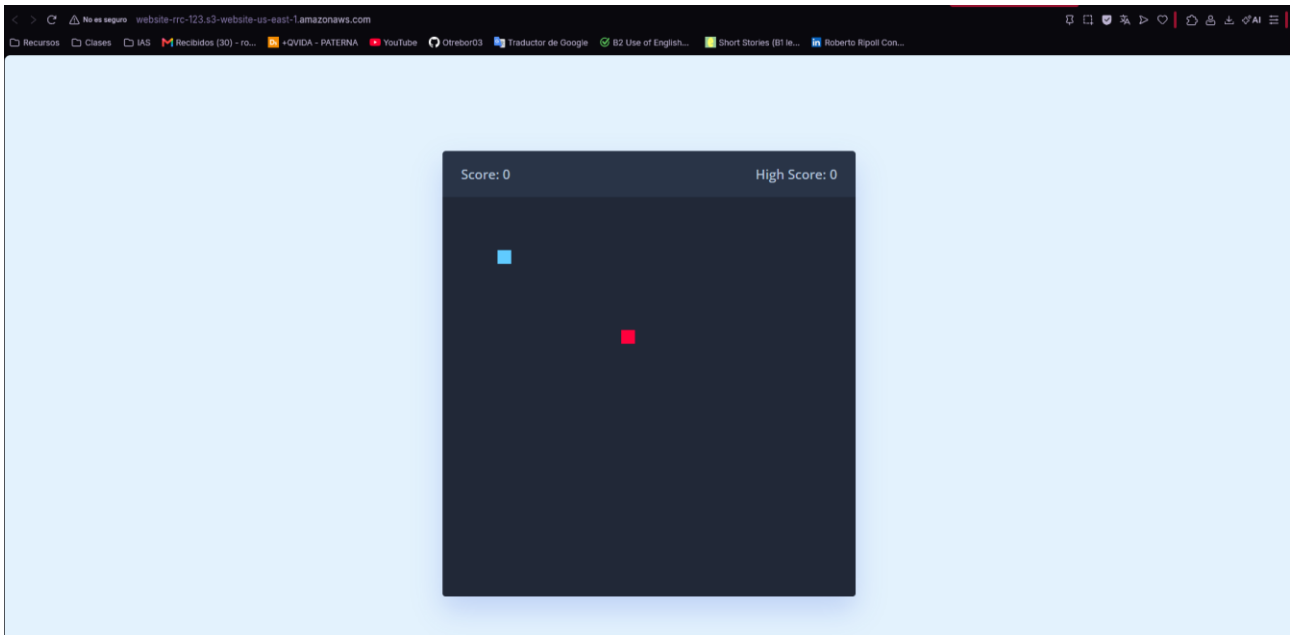
- **Principal**. Indica la entidad sobre la que aplican los permisos. En nuestro caso, al tratarse de un sitio web público, se indica un valor "*" para hacer referencia a cualquier usuario, incluidos usuarios anónimos.
- **Effect**. Puede indicarse las palabras "Allow" o "Deny", para especificar una autorización o denegación explícita de los permisos.
- **Action**. Indica el conjunto de llamadas a la API sobre el que se conceden o deniegan los permisos a la entidad especificada en **Principal**. En nuestro caso, sólo se desea que las entidades puedan recuperar objetos del bucket de S3.
- **Resource**. Indica los recursos sobre los que se conceden permisos. En nuestro caso, los recursos serán todos los objetos del bucket de Amazon S3 (por eso el ARN termina en `/*`).
- **Condition**. Es un atributo opcional que se utiliza para otorgar o denegar condicionalmente los permisos especificados en la sentencia, para conseguir un nivel de detalle mucho mayor.

Por último, pegamos la política anterior en la sección **Policy** y presionamos el botón **Save changes**.

Adaptado de J. E. Vera · CC BY-NC



16) Si a continuación, abrimos una nueva ventana del navegador en modo privado/incógnito y accedemos de nuevo a la URL del sitio web, podremos ver cómo nuestro sitio web estático es ahora accesible.



17) A partir de este punto, vamos a trabajar como lo haría un desarrollador: modificamos el sitio en local y lo volvemos a desplegar. En VS Code, abrimos el archivo *index.html*, cambiamos algún texto visible (por ejemplo, el contenido de la etiqueta `<title>`) y guardamos el archivo (**Ctrl+S**). Después, sincronizamos la carpeta con el bucket desde el terminal:

```
aws s3 sync . s3://<bucket>
```

A diferencia de `cp --recursive`, el comando `sync` sólo sube los archivos que han cambiado. Se puede comprobar qué haría antes de ejecutarlo añadiendo la opción `--dryrun`. Para ver el cambio, se recarga la página del sitio web en la ventana de incógnito (con **Ctrl+F5** para evitar la caché).

Adaptado de J. E. Vera · CC BY-NC

index.html

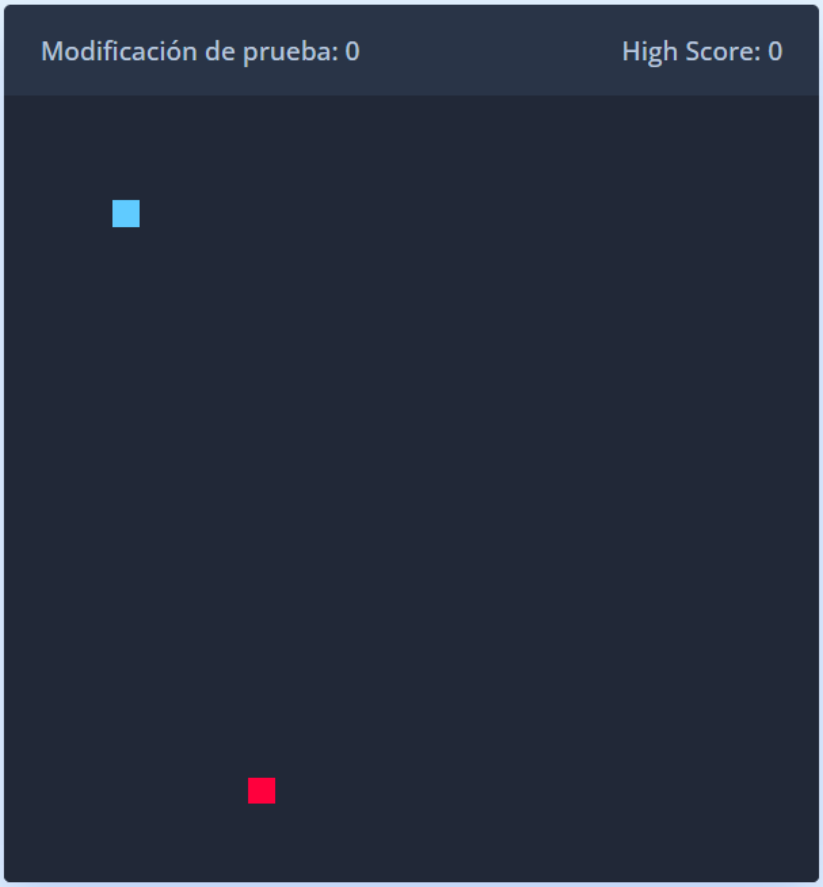
Administracion_BBDD_Almacenamiento > T1 - Almacenamiento de Objetos Con Amazon S3 > Practica 1.7 > aws-academy-fp-asir > resources > website-s3-cloud9 > index.html > html > head > title

```
<!DOCTYPE html>
<!-- Coding By CodingNepal - youtube.com/codingnepal -->
<html lang="en" dir="ltr">
  <head>
    <meta charset="utf-8">
    <title>Roberto Ripoll Cones</title>
    <link rel="stylesheet" href="style.css">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.3.0/css/all.min.css">
    <script src="script.js" defer></script>
  </head>
  <body>
    <div class="wrapper">
      <div class="game-details">
        <span class="score">Modificación de prueba: 0</span>
        <span class="high-score">High Score: 0</span>
      </div>
      <div class="play-board"></div>
      <div class="controls">
        <i data-key="ArrowLeft" class="fa-solid fa-arrow-left-long"></i>
        <i data-key="ArrowUp" class="fa-solid fa-arrow-up-long"></i>
        <i data-key="ArrowRight" class="fa-solid fa-arrow-right-long"></i>
        <i data-key="ArrowDown" class="fa-solid fa-arrow-down-long"></i>
      </div>
    </div>

  </body>
</html>
```

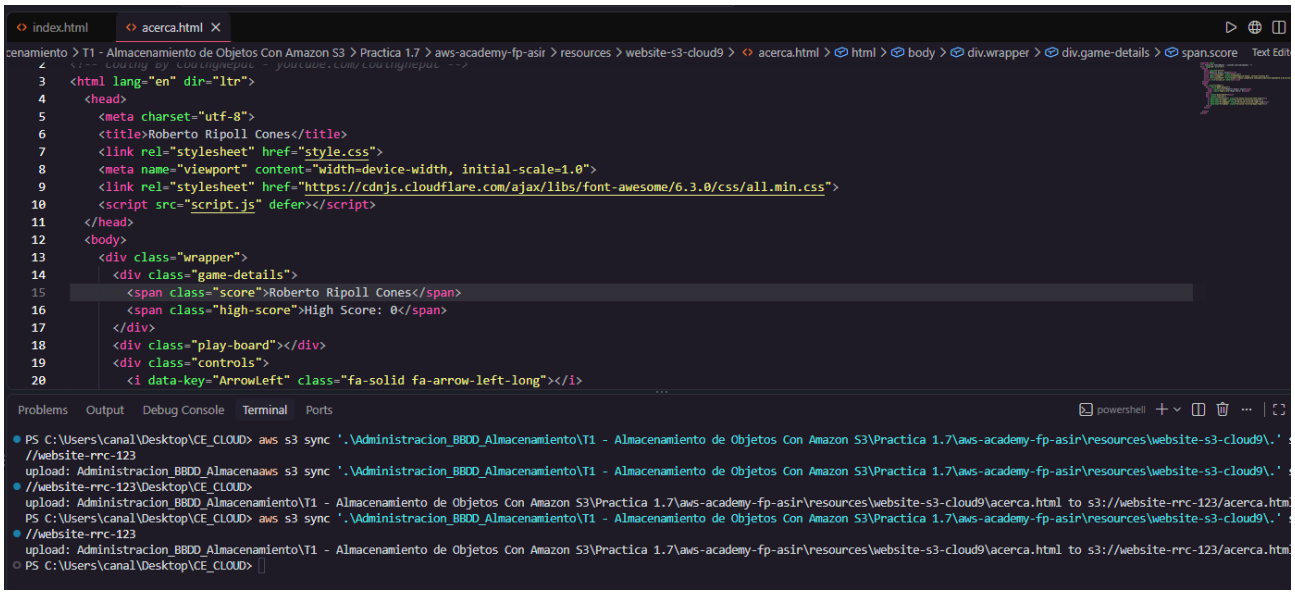
Problems Output Debug Console Terminal Ports

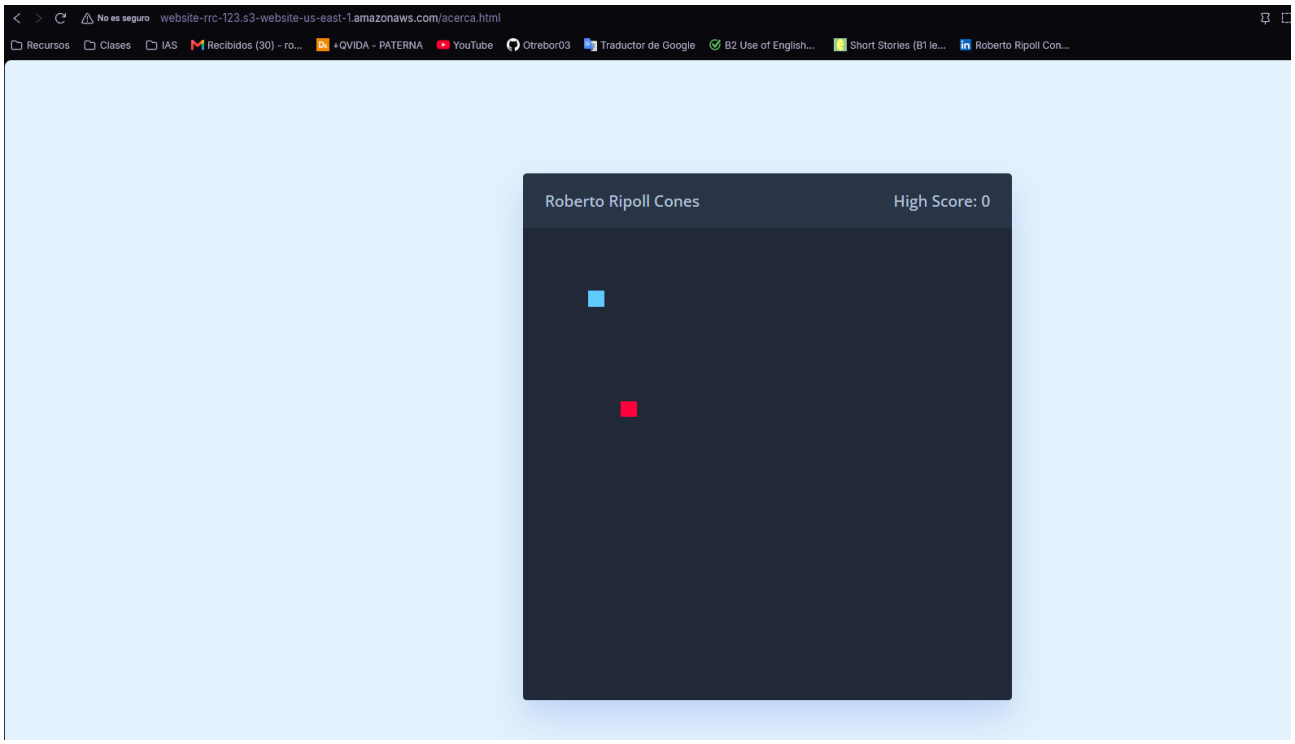
```
PS C:\Users\canal\Desktop\CE_CLOUD> aws s3 sync '.\Administracion_BBDD_Almacenamiento\T1 - Almacenamiento de Objetos Con Amazon S3\Practica 1.7\aws-academy-fp-asir\resources\website-s3-cloud9\.' s3://website-rrc-123
upload: Administracion_BBDD_Almacenamiento\T1 - Almacenamiento de Objetos Con Amazon S3\Practica 1.7\aws-academy-fp-asir\resources\website-s3-cloud9\index.html to s3://website-rrc-123/index.html
PS C:\Users\canal\Desktop\CE_CLOUD>
```



Adaptado de J. E. Vera · CC BY-NC

Reto: crea un nuevo archivo *acerca.html* con tu nombre, súbelo con `aws s3 sync` y accede a él desde el navegador añadiendo `/acerca.html` a la URL del sitio.



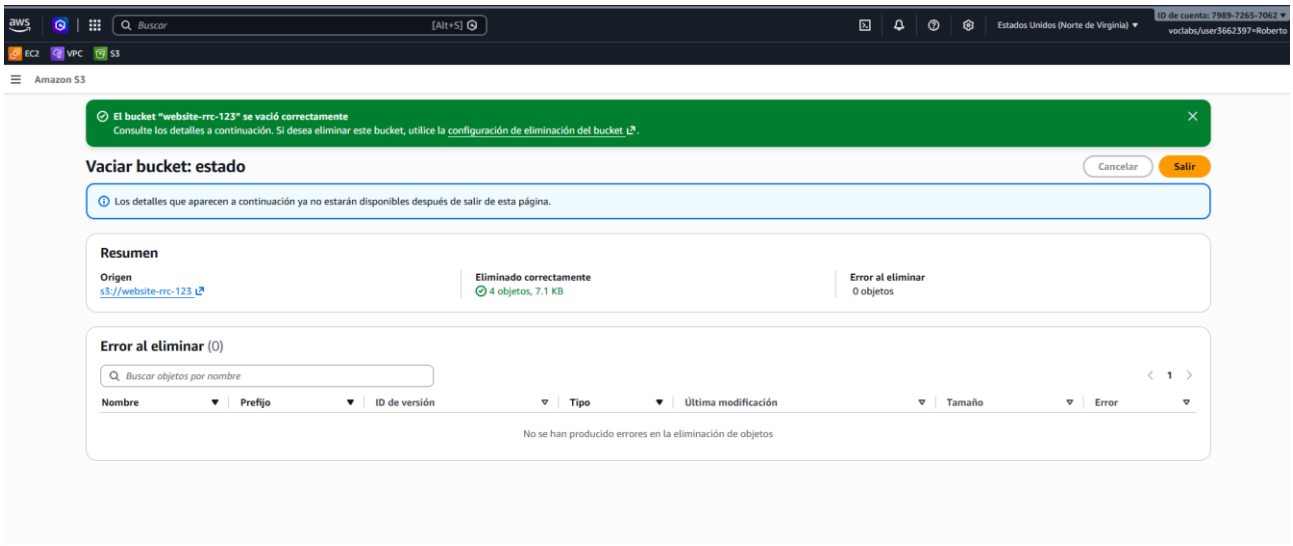


Adaptado de J. E. Vera · CC BY-NC

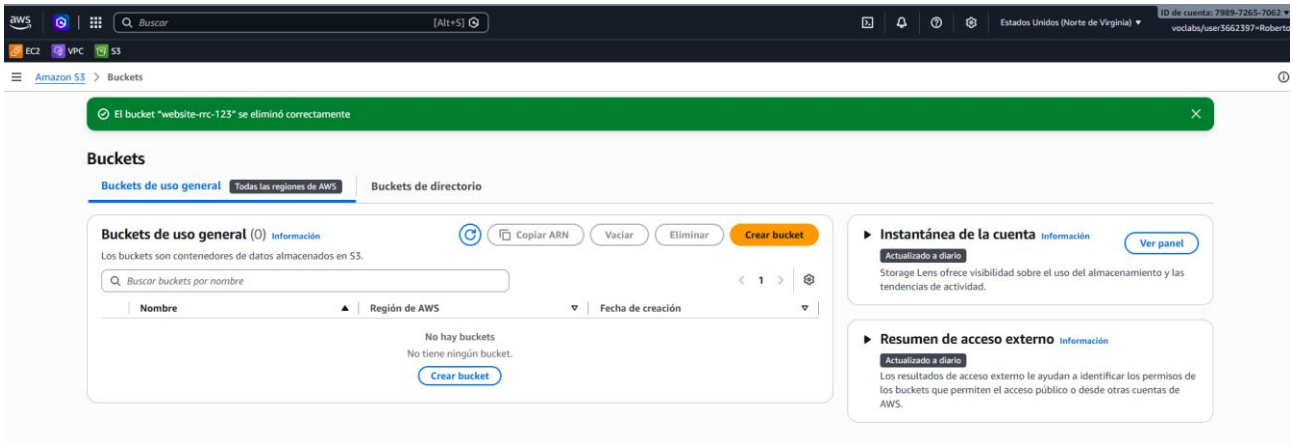
Limpieza de la Práctica:

Para terminar esta práctica y liberar los recursos creados, evitando así el consumo de créditos de AWS Academy Learner Labs, simplemente debemos dar los siguientes pasos:

- **Vaciar el bucket de Amazon S3**. Para ello, desde la consola de Amazon S3, seleccionamos la opción **Buckets** del menú lateral, marcamos el botón de radio de nuestro bucket y seleccionamos la opción **Empty**. Confirmamos nuestro deseo de vaciar el bucket escribiendo *permanently delete* en el campo de texto y pulsando **Empty**.



- **Eliminar el bucket de Amazon S3**. Una vez dado el paso anterior, marcamos de nuevo el botón de radio de nuestro bucket de S3 y seleccionamos la opción **Delete**, confirmando de nuevo la operación escribiendo el nombre del bucket.



- **Alternativa desde la terminal**: el comando `aws s3 rb s3://<bucket> --force` vacía y elimina el bucket en un solo paso.
- **Eliminar las credenciales del equipo**, sobre todo si es un ordenador compartido: se borra el fichero `credentials` (`Remove-Item $HOME\.aws\credentials` en PowerShell o `rm ~/.aws/credentials` en Linux/macOS).

Recuerda también que, para hacer un uso responsable de los recursos en la nube, **el laboratorio de AWS Academy debe cerrarse** presionando el botón *End Lab* desde la plataforma.

Adaptado de J. E. Vera · CC BY-NC

Resolución de problemas frecuentes:

Síntoma	Causa probable	Solución
`aws: command not found`	La AWS CLI no está instalada, o el terminal se abrió antes de instalarla.	Instalar la AWS CLI v2 y reiniciar VS Code.
`ExpiredToken`	Las credenciales proceden de una sesión anterior del laboratorio.	Repetir el paso 4 con las credenciales de la sesión actual.
`Unable to locate credentials`	El fichero credentials no está guardado, está en otra ubicación o falta la línea [default].	Revisar el paso 4 y repetir la comprobación del paso 5.
`BucketAlreadyExists`	El nombre del bucket ya existe en otra cuenta.	Elegir otro nombre (cambiar las iniciales o el número final).
Error al guardar la política (`BlockPublicPolicy`)	El bloqueo de acceso público sigue activado.	Permissions › Block public access › Edit: desmarcar Block all public access.
El sitio sigue dando 403 tras crear la política	Falta el `/*` al final del ARN o el navegador usa la caché.	Revisar el valor de Resource y recargar con Ctrl+F5 en incógnito.
Error 404 en el sitio web	Los archivos no están en el bucket o el Index document no se llama index.html.	Comprobar con `aws s3 ls s3://<bucket>` y revisar el paso 11.

Práctica adaptada a partir de «Amazon S3: Creación de un sitio web estático alojado en Amazon S3 utilizando AWS Cloud9», de José Emilio Vera (AWS Academy), publicada con licencia CC BY-NC. Se ha sustituido AWS Cloud9 por Visual Studio Code en local y la AWS CLI.